De-Platformed

Offline Mesh Messenger - Prototype v1.0

Technical Report and Faculty Demo Reference

Project Type: Mobile Application Prototype (Native Android)
Platform: React Native + Expo + Google Nearby Connections API
Build Output: Standalone Android APK (~218 MB)
Compiled: August 2026

This document covers the complete technical architecture, experimental setup, libraries used, feature definitions, and implementation details of the De-Platformed prototype - a 100% offline peer-to-peer (P2P) encrypted mesh messaging application that works without internet, SIM cards, or central servers.

1. Project Overview

De-Platformed is an offline-first, peer-to-peer (P2P) encrypted mesh communication system. It enables nearby Android devices to communicate directly over radio frequencies - Bluetooth Low Energy (BLE) and Wi-Fi Direct - without any internet connection, cellular tower, SIM card, or central server.

1.1 Problem Statement

Modern communication depends entirely on centralised infrastructure. When this fails during a disaster, power outage, or deliberate shutdown, people are left with no way to communicate. De-Platformed removes the infrastructure dependency entirely: every phone becomes both a messaging node and a relay node.

1.2 Scope of This Prototype

- Direct 1-hop encrypted P2P messaging between two Android devices.
- Group mesh broadcast chat for all connected peers.
- Private 1-on-1 encrypted chat rooms.
- Emergency Bulletin Board with categorised announcements (INFO / URGENT / RESOURCE).
- Device Trust System with persistent auto-reconnect for saved peers.
- Store-and-Forward CRDT queue for offline message delivery.
- Standalone self-contained Android APK - no PC or internet required to run.

2. System Architecture

The prototype is organised into four logical layers. Each layer has a single responsibility and communicates only with the layer immediately above or below it.

Layer	Name	Responsibility	Key File(s)
4	UI / App Screens	User-facing screens: Join, Chat, Bulletin, Mesh/Peers	JoinScreen.jsx ChatScreen.jsx BulletinScreen.jsx PeersScreen.jsx
3	Global State and Logic	Identity, peer map, send/receive logic, trust memory	AppContext.jsx
2	Security / Crypto	ECDH key generation, AES-256-GCM encrypt/decrypt	crypto.js
1a	Sync / CRDT	Persistent grow-only message store, offline queue, dedup by UUID	crdt.js
1b	P2P Transport	Nearby Connections API: discovery, connect, send, events	p2p.js NearbyModule.java

2.1 Architecture Diagram

```
LAYER 4 - UI (React Native Screens)
JoinScreen | ChatScreen | BulletinScreen | PeersScreen
| React Context (hooks/callbacks)
LAYER 3 - AppContext.jsx (Global State)
Identity . Peer Map . Trust Memory . Send/Recv Logic
| |
LAYER 2 - crypto.js LAYER 1a - crdt.js
ECDH P-256 + Chat Store . Bulletin
AES-256-GCM Offline Queue . Merge
|
LAYER 1b - P2P Transport
p2p.js (JS bridge) <-> NearbyModule.java (Native Java)
Google Nearby Connections API (BLE + Wi-Fi Direct)
```

3. Experimental Setup

3.1 Hardware Requirements

Component	Specification / Requirement
Development Machine	Windows 10/11 PC - used for coding, building the APK
Target Devices	2 or more Android smartphones (Android 8.0+, API level 26+)
Bluetooth Radio	Bluetooth LE 4.0+ (built into all modern Android phones)
Wi-Fi Radio	Wi-Fi Direct support (built into Android 4.0+)
Storage	~250 MB free on device (for APK install and runtime data)
Internet	NOT required at runtime - only needed during initial APK installation
USB Cable / ADB	Required only during development for deployment and debugging

3.2 Software Requirements (Development)

Tool / Platform	Version	Purpose
Node.js	18 or 20 LTS	JavaScript runtime for Metro bundler and npm
npm	9+	Package manager - installs all JS dependencies
Expo CLI	57.x	Project scaffolding and dev server
React Native	0.86.0	Cross-platform mobile framework
Android Studio	Giraffe / Hedgehog	Android SDK, Gradle, emulator
Android SDK	API 34 (Target), API 26 (Min)	Android build toolchain
Gradle	8.x	Native Android build system
Java JDK	17	Required by Gradle / Android native compilation
Git	Any recent	Version control

3.3 Build and Deployment Procedure

Clone the repository into the working directory.

Run: npm install (installs all JavaScript dependencies from package.json).

Run: expo run:android (compiles the native Android APK using Gradle).

Compiled APK is at: Deplatformed.apk in the root directory (~218 MB).

Install the APK on 2+ Android devices via ADB or direct file transfer.

Both devices must have Bluetooth and Wi-Fi enabled. No internet needed after install.

4. Technologies and Libraries Used

4.1 Core Framework

Library / Technology	Version	Role in the App
React Native	0.86.0	Core UI framework - renders native Android components from JavaScript
Expo	~57.0.8	Build toolchain, SDK access, dev server, APK packaging
React	19.2.3	Component state management, hooks (useState, useEffect, useContext)
@react-navigation/native	^7.3.13	Navigation container - wraps all screens in a navigation tree
@react-navigation/bottom-tabs	^7.18.13	Bottom tab bar (Chat / Bulletin / Mesh tabs)

4.2 Networking and P2P Transport

Library / API	Version	Role in the App
Google Nearby Connections API (play-services-nearby:19.3.0)	Native Java	Core P2P radio API. Handles BLE advertisement, Wi-Fi Direct session creation, c
NearbyModule.java	Custom Native	Bridge between the React Native JS layer and the Nearby Connections Java SDK
react-native-ble-plx	^3.5.1	Supplementary BLE scanning library for additional Bluetooth device discovery.
react-native-wifi-p2p	^3.6.1	Wi-Fi Direct P2P library - provides direct device-to-device Wi-Fi connection primit

4.3 Cryptography and Security

Library	Version	Algorithm / Role
react-native-quick-crypto	^1.1.6	Native Web Crypto API implementation. Used for ECDH P-256 key generation ar
react-native-get-random-values	^2.0.0	Polyfill for crypto.getRandomValues() - generates cryptographically secure IVs ar
react-native-nitro-modules	^0.36.1	High-performance C++ bridge required by react-native-quick-crypto for native exe
buffer	^6.0.3	Node.js Buffer polyfill - converts raw crypto key bytes to/from Base64 for transmis

4.4 Storage and Data Persistence

Library	Version	Role
@react-native-async-storage/async-storage	^3.1.1	Persistent key-value store on the device. Stores: chat messages (dp_

5. Feature Definitions

F-01 User Identity and Onboarding

User enters a display name (alias) on first launch via JoinScreen.jsx.

A random emoji avatar is auto-assigned from a predefined set.

The app generates an ECDH P-256 key pair on-device. Private key never leaves the device.

Source: JoinScreen.jsx, AppContext.jsx - initializeApp()

F-02 Automatic P2P Discovery (BLE + Wi-Fi Direct)

App simultaneously advertises and scans for nearby devices using Nearby Connections API.

No pairing code or QR scan required - discovery is fully automatic.

Discovered devices appear in the Mesh (Peers) screen automatically.

Source: p2p.js - startDiscovery(), NearbyModule.startAdvertising()

F-03 Trust-Based Connection and Pairing Prompt

When a new device requests a connection, a native Android alert appears with Accept and Reject options.

Trusted peers are saved permanently to AsyncStorage on-device.

Previously trusted peers auto-reconnect silently with zero interaction required.

Source: App.js - onConnectionRequest handler, AppContext.jsx - addTrustedPeer()

F-04 ECDH Key Exchange and Session Encryption

Upon connection, both devices exchange public keys via HELLO and KEY message flow.

ECDH P-256 Diffie-Hellman derives a unique shared secret per peer pair.

All messages are encrypted with AES-256-GCM using a fresh 12-byte random IV per message.

Source: crypto.js - initCrypto(), exportPublicKey(), deriveSecret(), encrypt(), decrypt()

F-05 Group Mesh Chat (Broadcast)

Messages in the Chat screen are broadcast to ALL currently connected peers.

Each message is individually encrypted for each peer using that peer's AES session key.

Messages are persisted via the CRDT store and survive app restarts.

Source: ChatScreen.jsx, AppContext.jsx - sendChatMessage(text, null)

F-06 Private 1-on-1 Direct Chat

Tap Chat next to any connected peer in the Mesh screen to open a private room.

Messages filtered by targetId so only the intended recipient sees them.

Encrypted using the peer-specific AES-256-GCM session key.

Source: PeersScreen.jsx - private chat modal, AppContext.jsx - sendChatMessage(text, peerId)

F-07 Emergency Bulletin Board

Users post announcements categorised as: INFO, URGENT, or RESOURCE.

Posts are broadcast to all connected peers and stored in the CRDT bulletin store.

Bulletin posts survive offline and restart - the board is fully persistent.

Source: BulletinScreen.jsx, AppContext.jsx - sendBulletin()

F-08 Store-and-Forward Offline Queue (CRDT)

If no peers are in range, messages are queued in local AsyncStorage.

Queue drains automatically the moment a peer with a shared key reconnects.

CRDT design deduplicates by UUID and merges messages in any order without conflicts.

Source: crdt.js - queuePush(), queueDrain(), addChat(), addBulletin()

F-09 Peer Management and Trust Control

Connected peers shown with name, E2E encryption badge, and connection status.

Per-peer actions: Private Chat, Disconnect, Forget (Unpair).

Reset Saved Friends button clears all trusted peers from AsyncStorage.

Source: PeersScreen.jsx, AppContext.jsx - removeTrustedPeer(), clearAllTrustedPeers()

F-10 Standalone Offline APK Build

App compiled into a self-contained APK: Deplatformed.apk (~218 MB).

Includes pre-bundled JavaScript (index.android.bundle) - no Metro bundler needed.

No PC, internet, or dev server required to run the installed app.

Artifact: Deplatformed.apk in the root directory of the project

6. End-to-End Encryption Flow

Every message is encrypted before leaving the device. The encryption uses standard industry algorithms identical to those used in Signal and WhatsApp.

Step	Action	Algorithm / API Used
1	Key Generation (on app join)	ECDH P-256 key pair generated via QuickCrypto.subtle.generateKey()
2	Public Key Exchange (on peer connect)	Public key exported as Base64, sent in HELLO/KEY protocol message over Nearby
3	Shared Secret Derivation (once per peer)	ECDH: both devices independently derive the same 256-bit AES key - no key ever tr
4	Message Encryption (per message)	AES-256-GCM with fresh 12-byte random IV per message - outputs {iv, ct} as Base6
5	Message Decryption (on receive)	Recipient uses their copy of the shared AES key plus the received IV to decrypt ciph
6	Fallback (no key)	If shared secret does not exist yet, message is sent as plaintext and queued for encr

7. Key Source Files Reference

File	Location	Description
App.js	root/	Entry point. Initialises P2P, handles pairing alert, renders bottom tab navigation.
AppContext.jsx	src/	Global React context. Manages identity, peers map, send/receive logic, trust me
p2p.js	src/lib/	JS bridge to Nearby Connections native module. Discovery, connect, send, eve
crypto.js	src/lib/	ECDH P-256 + AES-256-GCM implementation using react-native-quick-crypto.
crdt.js	src/lib/	Grow-only CRDT message store backed by AsyncStorage. Chat, bulletin, offline
JoinScreen.jsx	src/screens/	Onboarding screen. User sets alias, triggers crypto init and P2P discovery start.
ChatScreen.jsx	src/screens/	Group mesh broadcast chat. Polls CRDT store every 1 second for new messag
BulletinScreen.jsx	src/screens/	Emergency bulletin board. Categorised posts (INFO / URGENT / RESOURCE).
PeersScreen.jsx	src/screens/	Mesh view. Connected peers, discovered devices, private 1-on-1 chat modal.
NearbyModule.java	android/.../	Native Java module. Implements Nearby Connections SDK and bridges to JS.
Deplatformed.apk	root/	Compiled standalone Android APK (~218 MB). Ready to install and run offline.

8. Future Roadmap (Planned - Not in Current Prototype)

The following features are documented in the Meshline Architecture v2 specification and are planned for future development phases beyond this prototype.

Feature	Description	Priority
Multi-Hop Mesh Relay	Intermediate phones forward encrypted envelopes they cannot read. TTL-based loop prevention.	High
X3DH Explicit Pairing	Signal Protocol X3DH key exchange - users send and accept contact requests before a chat exis	High
Double Ratchet Encryption	Upgrade from static ECDH to per-message ratchet keys (same algorithm as Signal messenger).	High
LoRa Radio Transport	2-15 km range per hop using a ~USD 20 LoRa module (e.g. Meshtastic T-Beam) paired via Bluet	Medium
Ham Radio Transport	5-50 km+ range using AX.25/APRS protocol over a ham radio HT. Requires operator license.	Low
Admin Broadcast Posts	Cryptographically signed posts from admin identities, flood-relayed to the entire mesh.	Medium
Help Request / Location	User-triggered I need help / I am safe signal with optional approximate location sharing.	Medium
Hybrid Cloud Mode	Optional backend relay (encrypted envelopes only) with FCM push notifications. Mesh is fallback.	Low
iOS Support	Multipeer Connectivity framework for Apple devices. Same app logic, different transport.	Medium
Battery-Aware Relay	Relay participation auto-throttles when battery falls below a configurable threshold.	Low

9. Competitive Landscape

Three active real-world apps target the same offline mesh messaging space. The table below shows where De-Platformed is designed to go further in future phases.

Feature	Bridgefy	Briar	Bitchat	De-Platformed
Offline P2P Mesh	Yes	Yes	Yes	Yes (Current)
E2E Encryption	Yes	Yes	Yes	Yes (Current)
Standalone APK	Yes	Yes	Yes	Yes (Current)
Signed Admin Broadcasts	No	No	No	Planned
Offline Resource Map	No	No	No	Planned
Help / Check-in Signal	No	No	No	Planned
Hybrid Cloud Fallback	No	Tor-only	No	Planned
LoRa / Ham Radio Support	No	No	No	Planned
Battery-Aware Relay	No	No	No	Planned

10. Summary

De-Platformed is a working prototype of a fully offline, encrypted peer-to-peer mesh messaging application for Android. It demonstrates that two or more nearby phones can securely exchange messages without any external infrastructure using only their built-in Bluetooth and Wi-Fi radios.

Key achievements of this prototype:

- 100% offline operation - no internet, no SIM, no server required.
- Industry-standard end-to-end encryption (ECDH P-256 + AES-256-GCM).
- Group mesh broadcast chat and private 1-on-1 encrypted direct messaging.
- Emergency Bulletin Board for categorised community announcements.
- Persistent trust memory - approved friends auto-reconnect silently.
- Store-and-Forward CRDT queue - messages delivered when peers come back in range.
- Deployed as a fully self-contained installable Android APK (~218 MB).

The architecture is intentionally designed for extensibility. The Transport layer is abstracted behind a generic interface, meaning future radios such as LoRa, Ham Radio, or internet cloud relay can be added without touching the encryption or sync layers.

End of Technical Report
De-Platformed Prototype v1.0 - August 2026